

Metodología de Sistemas II

Trabajo Práctico 2 Adapter

Farias, Gustavo

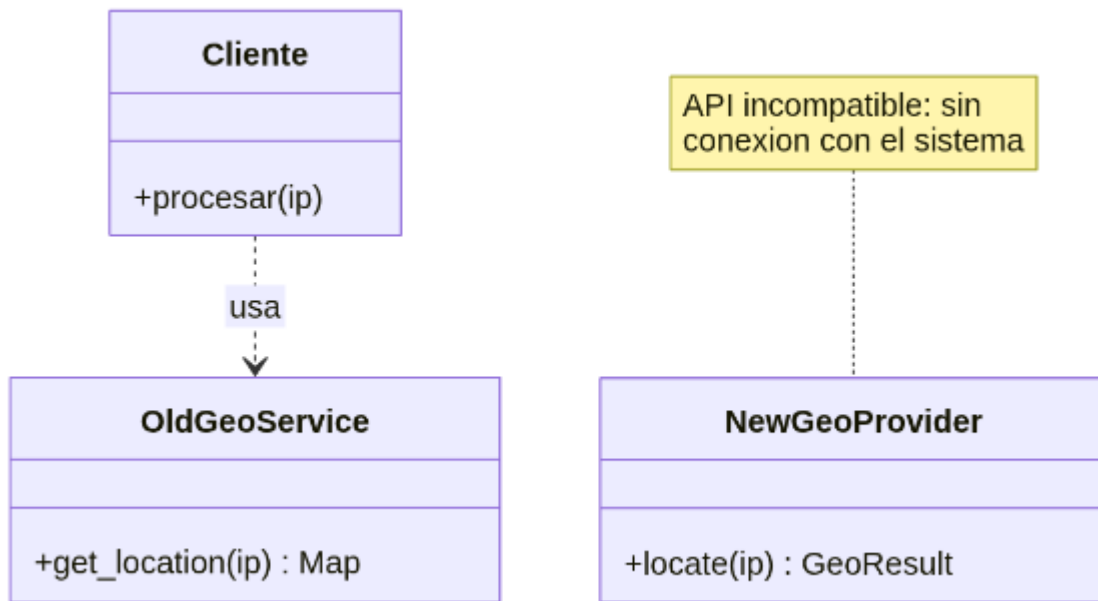
Matrícula: 101662

Comisión: M2025-13

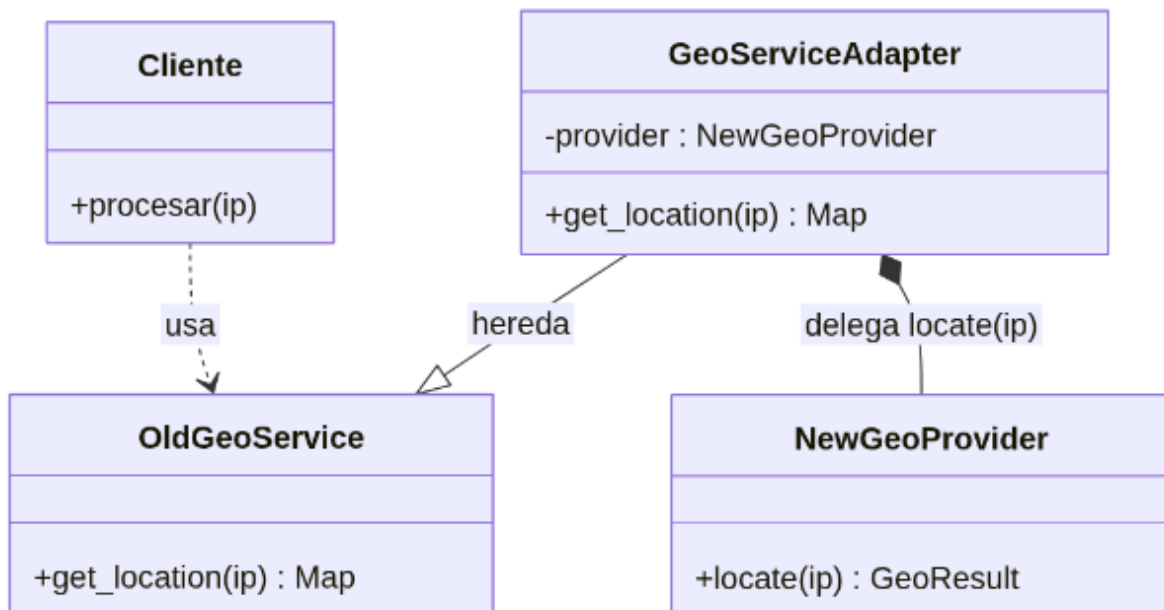
Repositorio GitHub: [Lucenear/Tecnicatura_Programacion_UTN](https://github.com/Lucenear/Tecnicatura_Programacion_UTN)

Diagrama UML

UML antes:



UML despues:



En el caso de que mañana llega un tercer proveedor, lo único que tendría que cambiar es mi punto de traducción (reescribir el interior del adapter, o crear un adapter nuevo y modificar esa misma única línea de la raíz de composición) y los clientes volverían a quedar intactos. Y si con el tiempo los proveedores se multiplicaran, la evolución natural de mi diseño sería extraer una interfaz interna e inyectarla por constructor, para que cambiar de proveedor pase a ser un cambio de configuración y no de código.

Antes del Adapter:

Raíz de composición:

```
OldGeoService geo = new OldGeoService();
```

Cualquiera de los 40 archivos del sistema:

```
Map<String, Object> data = geo.get_location("200.45.123.10");  
System.out.println(data.get("city") + ", " + data.get("lat"));
```

En este caso, el sistema entero dependía de una librería que quedó sin mantenimiento. Y el reemplazo no podía enchufarse directamente, porque su API es incompatible con lo que los 40 archivos esperan.

El defecto no estaba en la lógica de negocio (que funciona bien): estaba en que el punto de creación del servicio dejaba al sistema atado a una implementación concreta con un formato de respuesta específico. Cuando esa implementación murió, no había forma de sustituirla sin reescribir todo.

Despues del Adapter:

Raíz de composición:

```
OldGeoService geo = new GeoServiceAdapter();
```

Cualquiera de los 40 archivos del sistema (siguen iguales):

```
Map<String, Object> data = geo.get_location("200.45.123.10");  
System.out.println(data.get("city") + ", " + data.get("lat"));
```

Por dentro, el Adapter es el traductor que hace posible que ese código siga funcionando:

```
@Override  
public Map<String, Object> get_location(String ip) {  
    GeoResult r = provider.locate(ip);           // llama al nuevo  
    proveedor  
    Map<String, Object> data = new HashMap<>(); // traduce al  
    formato viejo  
    data.put("lat", r.getCoordinates().getLatitude());  
    data.put("lng", r.getCoordinates().getLongitude());  
}
```

```
data.put("city", r.getAddress().getLocality());  
data.put("country", r.getAddress().getNation());  
return data;  
}
```

Descarte refactorizar masivamente los 40 archivos porque implicaba tocar lógica que funciona, con un riesgo de regresión desproporcionado para un cambio de infraestructura. Elegi Adapter porque era la solución mínima: una única clase nueva que por fuera replica el contrato legacy (OldGeoService) y por dentro compone un NewGeoProvider traduciendo su respuesta. Con eso reduje la migración a una sola línea en la raíz de composición, el cliente quedó idéntico y el punto de cambio aislado: si llega un tercer proveedor, solo se toca mi adapter.